

TSwapPool Security Audit Report

Prepared by: Shweta Paranjape

Date: April 17, 2026

Contracts: TSwapPool.sol, PoolFactory.sol

Language: Solidity 0.8.20

License: GNU General Public License v3.0

Summary

ID	Title	Severity
H-01	<code>deposit()</code> ignores the <code>deadline</code> parameter, allowing stale transactions to execute	High
H-02	Incorrect fee multiplier in <code>getInputAmountBasedOnOutput()</code> charges users ~90% fee	High
H-03	<code>_swap()</code> transfers extra tokens every 10 swaps, breaking the constant product invariant	High
H-04	<code>sellPoolTokens()</code> passes <code>poolTokenAmount</code> as output amount instead of input amount	High
M-01	<code>LiquidityAdded</code> event emits arguments in wrong order	Medium
M-02	<code>swapExactOutput()</code> has no slippage protection (<code>maxInputAmount</code> missing)	Medium
L-01	<code>swapExactInput()</code> return value is never assigned, silently returning zero to callers	Low
L-02	<code>totalLiquidityTokenSupply()</code> should be marked <code>external</code> instead of <code>public</code>	Low
L-03	Magic numbers used throughout without named constants	Low
I-01	No zero-address check in <code>TSwapPool</code> constructor	Informational
I-02	<code>swapExactInput()</code> and <code>swapExactOutput()</code> could be marked <code>external</code>	Informational
I-03	<code>liquidityTokensToMint</code> assignment after interaction in initial deposit branch	Informational
I-04	<code>swapExactOutput()</code> is missing <code>@param deadline</code> in NatSpec	Informational
I-05	<code>swapExactInput()</code> is missing NatSpec entirely	Informational
PoolFactory.sol		

ID	Title	Severity
L-04	<code>liquidityTokenSymbol</code> is constructed using <code>.name()</code> instead of <code>.symbol()</code>	Low
I-06	<code>PoolFactory__PoolDoesNotExist</code> error is declared but never used	Informational
I-07	No zero-address check on <code>wethToken</code> in <code>PoolFactory</code> constructor	Informational
I-08	<code>createPool()</code> does not validate that <code>tokenAddress</code> is a contract	Informational

High Severity

[H-01] `deposit()` ignores the `deadline` parameter, allowing stale transactions to execute

Description:

The `deposit()` function accepts a `deadline` parameter but never applies the `revertIfDeadlinePassed` modifier. As a result, transactions that sit in the mempool for an arbitrarily long time — such as during periods of network congestion — will still execute even if market conditions have changed dramatically since submission.

```
function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline // @audit deadline is accepted but never enforced
)
    external
    revertIfZero(wethToDeposit)
    returns (uint256 liquidityTokensToMint)
```

Impact:

A liquidity provider could have their deposit executed at an unfavorable pool ratio long after they intended, resulting in a loss of funds due to stale pricing. The `deadline` parameter gives users a false sense of protection while offering none in practice.

Proof of Concept:

The following Foundry test demonstrates that `deposit()` succeeds even when the deadline has already passed.

```
function testDepositDeadlineNotEnforced() public {
    // Warp time forward so the deadline is already in the past
    vm.warp(block.timestamp + 1 days);

    uint64 expiredDeadline = uint64(block.timestamp - 1); // already
```

passed

```

    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);

    // This should revert due to an expired deadline – but it does NOT
    // demonstrating the deadline parameter is completely ignored
    pool.deposit(
        100e18,
        0,
        100e18,
        expiredDeadline
    );
    vm.stopPrank();
}

```

Recommended Mitigation:

Apply the `revertIfDeadlinePassed` modifier to the `deposit()` function:

```

function deposit(
    uint256 wethToDeposit,
    uint256 minimumLiquidityTokensToMint,
    uint256 maximumPoolTokensToDeposit,
    uint64 deadline
)
    external
    revertIfZero(wethToDeposit)
    revertIfDeadlinePassed(deadline)
    returns (uint256 liquidityTokensToMint)

```

[H-02] Incorrect fee multiplier in `getInputAmountBasedOnOutput()` charges users approximately 90% in fees

Description:

The `getInputAmountBasedOnOutput()` function uses a multiplier of `10000` in its numerator instead of the correct `1000` used in `getOutputAmountBasedOnInput()`. Since the AMM applies a 0.3% fee using a 997/1000 ratio, the numerator multiplier must be `1000` to stay consistent. Using `10000` inflates the required input amount by a factor of 10, effectively charging users roughly 90% on every `swapExactOutput`-based trade.

```

// Incorrect – 10000 should be 1000
return
    ((inputReserves * outputAmount) * 10000) /
    ((outputReserves - outputAmount) * 997);

```

Impact:

Any user calling `swapExactOutput()` or `sellPoolTokens()` (which delegates to `swapExactOutput()`) will be charged approximately 10x the correct input amount, resulting in catastrophic financial loss on every such swap.

Proof of Concept:

The following Foundry test demonstrates the inflated input amount charged versus the expected amount.

```
function testGetInputAmountBasedOnOutputOvercharges() public {
    // Setup: 1000 WETH and 1000 POOL in reserves
    uint256 inputReserves = 1000e18;
    uint256 outputReserves = 1000e18;
    uint256 outputAmount = 100e18; // user wants 100 WETH out

    uint256 actualInput = pool.getInputAmountBasedOnOutput(
        outputAmount,
        inputReserves,
        outputReserves
    );

    // Correct formula result (multiplier = 1000): ~111 POOL
    uint256 expectedInput = ((inputReserves * outputAmount) * 1000) /
        ((outputReserves - outputAmount) * 997);

    // Actual result is ~10x higher due to the 10000 multiplier bug
    assertGt(actualInput, expectedInput * 9, "Input should be ~10x
    overcharged");
    console.log("Expected input:", expectedInput);
    console.log("Actual input: ", actualInput);
}
```

Recommended Mitigation:

Replace `10000` with `1000`:

```
return
    ((inputReserves * outputAmount) * 1000) /
    ((outputReserves - outputAmount) * 997);
```

[H-03] `_swap()` transfers extra tokens to the caller every 10 swaps, breaking the constant product invariant

Description:

The `_swap()` function contains a bonus mechanism that, every 10 swaps, transfers `1e18` tokens directly to the caller. This transfer is not accounted for by the AMM's constant product formula ($x * y = k$), directly draining the pool's reserves without any corresponding deposit.

```

swap_count++;
if (swap_count >= SWAP_COUNT_MAX) {
    swap_count = 0;
    outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000); //
breaks invariant
}

```

Impact:

Every 10 swaps, pool reserves are drained by **1e18** tokens for free. An attacker can repeatedly trigger this bonus with minimal-cost swaps, systematically draining the pool. Liquidity providers will suffer unrecoverable losses as their share of the pool decreases with each cycle.

Proof of Concept:

The following Foundry test demonstrates that the invariant $x * y = k$ is broken after 10 swaps.

► Details

****Proof of Code:****

```

function testInvariantBroken() public {
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 100e18, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 outputWeth = 1e17;

    vm.startPrank(user);
    poolToken.approve(address(pool), type(uint256).max);
    poolToken.mint(user, 100e18);
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    pool.swapExactOutput(poolToken, weth, outputWeth,
uint64(block.timestamp));
    vm.stopPrank();
}

```

```

    int256 startingY = int256(weth.balanceOf(address(pool)));
    int256 expectedDeltaY = int256(-1) * int256(outputWeth);

    uint256 endingY = weth.balanceOf(address(pool));
    int256 actualDeltaY = int256(endingY) - int256(startingY);
    assertEq(actualDeltaY, expectedDeltaY);
}

```

```

function testSwapBonusDrainsPoolAndBreaksInvariant() public {
    // Seed pool with 100 WETH and 100 POOL tokens
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 0, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    uint256 kBefore = weth.balanceOf(address(pool)) *
poolToken.balanceOf(address(pool));

    // Attacker performs 10 small swaps to trigger the bonus
    address attacker = makeAddr("attacker");
    vm.startPrank(attacker);
    weth.mint(attacker, 10e18);
    weth.approve(address(pool), type(uint256).max);

    for (uint256 i = 0; i < 10; i++) {
        pool.swapExactInput(weth, 1e15, poolToken, 0,
uint64(block.timestamp));
    }
    vm.stopPrank();

    uint256 kAfter = weth.balanceOf(address(pool)) *
poolToken.balanceOf(address(pool));

    // k should be constant – but it has dropped because 1e18 tokens were
taken for free
    assertLt(kAfter, kBefore, "Invariant k was broken by the bonus
transfer");
    console.log("k before:", kBefore);
    console.log("k after: ", kAfter);
}

```

Recommended Mitigation:

Remove the bonus incentive mechanism entirely. If a rewards program is desired, it must be funded from a separately allocated treasury external to the pool reserves:

```

// Remove entirely:
- swap_count++;

```

```
- if (swap_count >= SWAP_COUNT_MAX) {  
-     swap_count = 0;  
-     outputToken.safeTransfer(msg.sender, 1_000_000_000_000_000_000);  
- }
```

Also remove the `swap_count` and `SWAP_COUNT_MAX` state variables.

[H-04] `sellPoolTokens()` passes `poolTokenAmount` as the output amount instead of the input amount, causing incorrect swap execution

Description:

`sellPoolTokens()` is intended to allow users to sell a specific quantity of pool tokens in exchange for WETH. However, it calls `swapExactOutput()` and passes `poolTokenAmount` as the `outputAmount` argument. `swapExactOutput()` interprets this as the exact amount of WETH the user wants to *receive*, not the amount of pool tokens to *sell*. The function is therefore using the wrong swap interface entirely.

```
function sellPoolTokens(uint256 poolTokenAmount) external returns (uint256  
wethAmount) {  
    return swapExactOutput(  
        i_poolToken,  
        i_wethToken,  
        poolTokenAmount, // treated as WETH output amount, not pool token  
input  
        uint64(block.timestamp)  
    );  
}
```

Impact:

Users calling `sellPoolTokens()` will not sell the amount of pool tokens they specified. The protocol instead attempts to deliver `poolTokenAmount` of WETH, pulling however many pool tokens are required — potentially far more than the user intended. This results in users losing significantly more pool tokens than expected.

Proof of Concept:

The following Foundry test demonstrates that calling `sellPoolTokens(10e18)` drains far more than 10e18 pool tokens from the user.

```
function testSellPoolTokensUsesWrongAmount() public {  
    // Seed pool  
    vm.startPrank(liquidityProvider);  
    weth.approve(address(pool), 100e18);  
    poolToken.approve(address(pool), 100e18);  
    pool.deposit(100e18, 0, 100e18, uint64(block.timestamp));  
    vm.stopPrank();  
  
    address user = makeAddr("user");  
    poolToken.mint(user, 50e18);
```

```
vm.startPrank(user);
poolToken.approve(address(pool), type(uint256).max);

uint256 poolTokenBalanceBefore = poolToken.balanceOf(user);

// User intends to sell 10e18 pool tokens
pool.sellPoolTokens(10e18);

uint256 poolTokenBalanceAfter = poolToken.balanceOf(user);
uint256 poolTokensSpent = poolTokenBalanceBefore -
poolTokenBalanceAfter;

// The function treated 10e18 as WETH *output*, not pool token input.
// Far more than 10e18 pool tokens were pulled from the user.
assertGt(poolTokensSpent, 10e18, "More pool tokens were taken than the
user intended to sell");
console.log("Pool tokens spent:", poolTokensSpent);
vm.stopPrank();
}
```

Recommended Mitigation:

Use `swapExactInput()` instead, which correctly treats the first amount as the input token quantity:

```
function sellPoolTokens(
    uint256 poolTokenAmount,
    uint256 minWethToReceive
) external returns (uint256 wethAmount) {
    return swapExactInput(
        i_poolToken,
        poolTokenAmount,
        i_wethToken,
        minWethToReceive,
        uint64(block.timestamp)
    );
}
```

Medium Severity

[M-01] `LiquidityAdded` event emits `wethDeposited` and `poolTokensDeposited` in reversed order

Description:

In `_addLiquidityMintAndTransfer()`, the `LiquidityAdded` event is emitted with `poolTokensToDeposit` as the second argument and `wethToDeposit` as the third. The event definition declares `wethDeposited` second and `poolTokensDeposited` third. The two values are emitted in reversed order.


```
// Event definition:
event LiquidityAdded(address indexed liquidityProvider, uint256
wethDeposited, uint256 poolTokensDeposited);

// Incorrect emission (arguments are swapped):
emit LiquidityAdded(msg.sender, poolTokensToDeposit, wethToDeposit);
```

Impact:

All off-chain systems consuming this event — including subgraphs, indexers, accounting tools, and protocol front-ends — will record and display inverted deposit values. This misleads liquidity providers and produces incorrect TVL and analytics data.

Proof of Concept:

The following Foundry test uses `vm.expectEmit` to assert the correct event argument ordering and demonstrates that the contract emits them reversed.

```
function testLiquidityAddedEventArgsAreReversed() public {
    uint256 wethAmount      = 10e18;
    uint256 poolTokenAmount = 10e18;

    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), wethAmount);
    poolToken.approve(address(pool), poolTokenAmount);

    // Expect: LiquidityAdded(liquidityProvider, wethDeposited=10e18,
    poolTokensDeposited=10e18)
    vm.expectEmit(true, true, true, true);
    emit TSwapPool.LiquidityAdded(liquidityProvider, wethAmount,
    poolTokenAmount);

    // This will FAIL because the contract emits (liquidityProvider,
    poolTokenAmount, wethAmount)
    pool.deposit(wethAmount, 0, poolTokenAmount, uint64(block.timestamp));
    vm.stopPrank();
}
```

Recommended Mitigation:

Correct the argument order in the emit statement:

```
emit LiquidityAdded(msg.sender, wethToDeposit, poolTokensToDeposit);
```

[M-02] `swapExactOutput()` lacks a `maxInputAmount` parameter, exposing users to unbounded slippage and MEV sandwich attacks

Description:

`swapExactOutput()` allows users to specify exactly how many output tokens they want, but provides no mechanism to cap how many input tokens they are willing to spend. Under volatile market conditions or a MEV sandwich attack, the actual input required can be arbitrarily larger than the user anticipated at the time of signing.

```
function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
    // uint256 maxInputAmount  <-- missing
    uint64 deadline
) ...
```

Impact:

Users have zero slippage protection when using `swapExactOutput()`. A sandwich attacker can front-run the transaction to inflate prices, causing the victim to pay an unbounded amount of input tokens.

Proof of Concept:

The following Foundry test simulates a sandwich attack: an attacker front-runs the victim's swap by depositing a large amount, inflating the price. The victim ends up paying far more input tokens than anticipated, with no revert to protect them.

```
function testSwapExactOutputNoSlippageProtection() public {
    // Seed pool: 100 WETH, 100 P00L
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 0, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    // Victim wants exactly 10 P00L tokens out, paying WETH
    address victim = makeAddr("victim");
    weth.mint(victim, 100e18);
    vm.startPrank(victim);
    weth.approve(address(pool), type(uint256).max);

    // Calculate expected input before the attack
    uint256 expectedInput = pool.getInputAmountBasedOnOutput(
        10e18,
        weth.balanceOf(address(pool)),
        poolToken.balanceOf(address(pool))
    );

    // Attacker front-runs: dumps a huge amount of WETH into the pool,
    // skewing the ratio
    address attacker = makeAddr("attacker");
    weth.mint(attacker, 1000e18);
    vm.startPrank(attacker);
```

```

    weth.approve(address(pool), type(uint256).max);
    pool.swapExactInput(weth, 1000e18, poolToken, 0,
uint64(block.timestamp));
    vm.stopPrank();

    // Victim's swap now executes at the manipulated price – no
    maxInputAmount check saves them
    uint256 victimWethBefore = weth.balanceOf(victim);
    vm.startPrank(victim);
    pool.swapExactOutput(weth, poolToken, 10e18, uint64(block.timestamp));
    uint256 victimWethAfter = weth.balanceOf(victim);
    vm.stopPrank();

    uint256 actualInput = victimWethBefore - victimWethAfter;

    // Victim paid far more than the pre-attack expected input
    assertGt(actualInput, expectedInput * 2, "Victim overpaid due to lack
of slippage protection");
    console.log("Expected input (pre-attack):", expectedInput);
    console.log("Actual input paid by victim:", actualInput);
}

```

Recommended Mitigation:

Add a `maxInputAmount` parameter and revert if the computed input exceeds it:

```

function swapExactOutput(
    IERC20 inputToken,
    IERC20 outputToken,
    uint256 outputAmount,
    uint256 maxInputAmount,
    uint64 deadline
) public revertIfZero(outputAmount) revertIfDeadlinePassed(deadline)
returns (uint256 inputAmount) {
    inputAmount = getInputAmountBasedOnOutput(outputAmount, inputReserves,
outputReserves);

    if (inputAmount > maxInputAmount) {
        revert TSwapPool__InputTooHigh(inputAmount, maxInputAmount);
    }

    _swap(inputToken, inputAmount, outputToken, outputAmount);
}

```

Low Severity

[L-01] `swapExactInput()` return value is never assigned, silently returning zero to callers

Description:

`swapExactInput()` declares a named return variable `output` but never assigns it. The internal `_swap()` call is `void`. Any caller relying on this return value will silently receive `0`.

```
returns (uint256 output) // declared but never assigned
{
    ...
    _swap(inputToken, inputAmount, outputToken, outputAmount);
    // 'output' is implicitly 0
}
```

Impact:

Any contract or off-chain integration calling `swapExactInput()` and using its return value for further logic (e.g., slippage checks, accounting) will silently operate on zero, potentially leading to integration bugs or downstream fund loss in composed protocols.

Proof of Concept:

```
function testSwapExactInputReturnsZero() public {
    // Seed pool
    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 100e18);
    poolToken.approve(address(pool), 100e18);
    pool.deposit(100e18, 0, 100e18, uint64(block.timestamp));
    vm.stopPrank();

    address user = makeAddr("user");
    weth.mint(user, 1e18);
    vm.startPrank(user);
    weth.approve(address(pool), type(uint256).max);

    uint256 returnedOutput = pool.swapExactInput(
        weth,
        1e18,
        poolToken,
        0,
        uint64(block.timestamp)
    );
    vm.stopPrank();

    // Despite a successful swap, the returned value is 0
    assertEq(returnedOutput, 0, "Return value should be non-zero but is
0");
}
```

Recommended Mitigation:

Assign the output amount before the function returns:

```
output = outputAmount;
```

[L-02] `totalLiquidityTokenSupply()` is marked `public` but is never called internally and should be `external`

Description:

`totalLiquidityTokenSupply()` is only ever called from outside the contract. Marking it `public` rather than `external` incurs unnecessary gas costs because the Solidity compiler generates additional code to handle potential internal calls.

Impact:

Minor but unnecessary gas overhead on every external call to this function.

Proof of Concept:

```
function testTotalLiquidityTokenSupplyVisibility() public {
    // This test exists to document the gas difference.
    // Mark the function external and run: forge snapshot --diff
    // to confirm reduced gas consumption on external calls.
    uint256 gasBefore = gasleft();
    pool.totalLiquidityTokenSupply();
    uint256 gasUsed = gasBefore - gasleft();
    console.log("Gas used (public):", gasUsed);
    // After changing to external, re-run to observe the reduction.
}
```

Recommended Mitigation:

```
function totalLiquidityTokenSupply() external view returns (uint256) {
    return totalSupply();
}
```

[L-03] Magic numbers `997`, `1000`, and `1_000_000_000_000_000_000` are used without named constants

Description:

Fee-related literals `997` and `1000` appear directly in arithmetic without explanation, as does the bonus amount `1_000_000_000_000_000_000` in `_swap()`. Unnamed literal values make code harder to audit, reason about, and safely modify.

Impact:

Reduced code clarity and auditability. Future maintainers may misinterpret or incorrectly modify these values, increasing the risk of introducing new vulnerabilities.

Proof of Concept:

```
function testFeeConstantsAreUndocumentedLiterals() public pure {
    // This test documents the expected fee model by deriving it from
    first principles.
    // The protocol applies a 0.3% fee, meaning 99.7% of input is used.
    // Correct scaling: 997 / 1000 = 0.997
    // The bug in getInputAmountBasedOnOutput uses 10000 instead of 1000,
    // which is only detectable if the reader knows these should be named
    constants.
    uint256 feeNumerator    = 997;
    uint256 feeDenominator = 1000;
    assertEq((feeDenominator - feeNumerator), 3); // 0.3% fee
}
```

Recommended Mitigation:

Define named constants at the contract level:

```
uint256 private constant FEE_NUMERATOR    = 997;
uint256 private constant FEE_DENOMINATOR = 1000;
uint256 private constant BONUS_AMOUNT    = 1e18;
```

[L-04] `liquidityTokenSymbol` is constructed using `.name()` instead of `.symbol()`, producing incorrect token symbols

Description:

In `createPool()`, both the liquidity token name and symbol are derived from `IERC20(tokenAddress).name()`. The symbol should use `.symbol()` instead. For example, a token with the name "Dai Stablecoin" and symbol "DAI" would produce a liquidity token symbol of "tsDai Stablecoin" rather than the correct "tsDAI".

```
string memory liquidityTokenName    = string.concat("T-Swap ",
IERC20(tokenAddress).name());
// @audit should call .symbol(), not .name()
string memory liquidityTokenSymbol = string.concat("ts",
IERC20(tokenAddress).name());
```

Impact:

All deployed pools will have malformed liquidity token symbols. Users holding liquidity tokens will see verbose and incorrect symbol strings in wallets, block explorers, and DEX interfaces. Off-chain integrations that look up tokens by symbol will fail to match them.

Proof of Concept:

```
function testCreatePoolUsesNameForSymbol() public {
    // Deploy a mock ERC20 with a distinct name and symbol
    MockERC20 token = new MockERC20("Dai Stablecoin", "DAI");

    address poolAddress = factory.createPool(address(token));
    TSwapPool createdPool = TSwapPool(poolAddress);

    string memory symbol = createdPool.symbol();
    string memory name    = createdPool.name();

    console.log("Liquidity token name: ", name);
    console.log("Liquidity token symbol:", symbol);

    // Correct: symbol should be "tsDAI"
    // Bug:      symbol is "tsDai Stablecoin"
    assertEq(symbol, "tsDAI", "Symbol should use .symbol(), not
.name());
    assertEq(name, "T-Swap Dai Stablecoin", "Name is correct");
}
```

Recommended Mitigation:

```
string memory liquidityTokenSymbol = string.concat("ts",
IERC20(tokenAddress).symbol());
```

Informational

[I-01] No zero-address validation in **TSwapPool** constructor for **poolToken** and **wethToken**

Description:

The constructor assigns **poolToken** and **wethToken** to immutable state variables without checking for the zero address. Deploying with a zero address would produce a permanently broken and non-upgradeable contract.

Proof of Concept:

```
function testConstructorAllowsZeroAddress() public {
    // Both of these deployments succeed with address(0) – they should
    revert
    TSwapPool brokenPool = new TSwapPool(
        address(0), // poolToken = zero address
        address(weth),
        "Broken LP",
        "BLP"
    );
    assertEq(brokenPool.getPoolToken(), address(0)); // silently broken
```

```
TSwapPool brokenPool2 = new TSwapPool(
    address(poolToken),
    address(0),    // wethToken = zero address
    "Broken LP2",
    "BLP2"
);
assertEq(brokenPool2.getWeth(), address(0)); // silently broken
}
```

Recommended Mitigation:

```
if (poolToken == address(0) || wethToken == address(0)) revert
TSwapPool__InvalidToken();
```

[I-02] `swapExactInput()` and `swapExactOutput()` are `public` but never called internally — mark them `external`

Description:

Neither function is invoked from within the contract. Marking them `external` avoids unnecessary ABI encoding overhead and signals their intended usage correctly.

Proof of Concept:

```
function testPublicFunctionsCostMoreGas() public {
    // Run: forge snapshot --diff after changing to external
    // to confirm reduced call overhead for both functions.
    uint256 g1 = gasleft();
    pool.swapExactInput(weth, 1e15, poolToken, 0,
    uint64(block.timestamp));
    uint256 gasPublic = g1 - gasleft();
    console.log("Gas (public swapExactInput):", gasPublic);
}
```

Recommended Mitigation:

Change both function visibilities to `external`.

[I-03] Initial deposit branch assigns `liquidityTokensToMint` after `_addLiquidityMintAndTransfer()`, violating CEI pattern

Description:

In the `else` branch of `deposit()`, tokens are transferred before `liquidityTokensToMint` is assigned. Although not currently exploitable given known ERC20 implementations, this violates the Checks-Effects-Interactions (CEI) pattern and may become a risk if the codebase is extended.

Proof of Concept:

```
function testInitialDepositViolatesCEI() public {
    // Illustrates the ordering: interaction
    (_addLiquidityMintAndTransfer) happens
    // before the effect (liquidityTokensToMint = wethToDeposit).
    // No totalLiquidityTokenSupply exists yet, so the else branch is
    taken.

    vm.startPrank(liquidityProvider);
    weth.approve(address(pool), 10e18);
    poolToken.approve(address(pool), 10e18);

    // If _addLiquidityMintAndTransfer triggered a reentrancy callback,
    // liquidityTokensToMint would still be 0 at the time of re-entry.
    uint256 lpTokens = pool.deposit(10e18, 0, 10e18,
    uint64(block.timestamp));
    vm.stopPrank();

    // Tokens are minted correctly in this non-reentrant case,
    // but the assignment ordering is still a CEI violation.
    assertEq(lpTokens, 10e18);
}
```

Recommended Mitigation:

Assign `liquidityTokensToMint = wethToDeposit` before calling `_addLiquidityMintAndTransfer()`.

[I-04] `swapExactOutput()` NatSpec is missing `@param deadline` documentation

Description:

The NatSpec block for `swapExactOutput()` documents `inputToken`, `outputToken`, and `outputAmount` but omits the `deadline` parameter entirely, reducing interface clarity for integrators.

Proof of Concept:

Inspecting the source NatSpec directly reveals the omission:

```
/*
 * @param inputToken ERC20 token to pull from caller
 * @param outputToken ERC20 token to send to caller
 * @param outputAmount The exact amount of tokens to send to caller
 * // @audit @param deadline is missing
 */
```

Recommended Mitigation:

Add the missing parameter documentation:

```
@param deadline The Unix timestamp by which the transaction must be
executed.
```

[I-05] `swapExactInput()` is missing NatSpec documentation entirely

Description:

Unlike other public-facing functions in the contract, `swapExactInput()` has no `@notice`, `@dev`, `@param`, or `@return` documentation, reducing readability and making the function harder to integrate against safely.

Proof of Concept:

The function header in source has no NatSpec block preceding it:

```
// @audit-info where is the natspec??
function swapExactInput(
    IERC20 inputToken,
    uint256 inputAmount,
    ...
```

Recommended Mitigation:

Add a complete NatSpec block:

```
/// @notice Swaps an exact amount of input tokens for as many output
tokens as possible.
/// @param inputToken The ERC20 token the caller is providing.
/// @param inputAmount The exact amount of inputToken to send.
/// @param outputToken The ERC20 token the caller wants to receive.
/// @param minOutputAmount The minimum acceptable amount of outputToken to
receive (slippage guard).
/// @param deadline The Unix timestamp by which the transaction must be
executed.
/// @return output The amount of outputToken received by the caller.
```

[I-06] `PoolFactory__PoolDoesNotExist` error is declared but never used, adding unnecessary bytecode

Description:

The custom error `PoolFactory__PoolDoesNotExist` is defined at the contract level but is never referenced or reverted anywhere in the contract. Dead code increases deployment gas costs and clutters the ABI. Its absence also means `getPool()` silently returns `address(0)` for non-existent pools rather than reverting with a helpful error.

```
// Declared but never used:
error PoolFactory__PoolDoesNotExist(address tokenAddress);
```

Proof of Concept:

```
function testGetPoolReturnsSilentZeroForNonexistentPool() public {
    address fakeToken = makeAddr("fakeToken");

    // Returns address(0) silently instead of reverting with
    PoolDoesNotExist
    address result = factory.getPool(fakeToken);
    assertEq(result, address(0), "Should return zero for non-existent
pool");
    // Callers that don't check for address(0) will silently operate on a
    null address
}
```

Recommended Mitigation:

Either remove the unused error, or actively use it in `getPool()`:

```
function getPool(address tokenAddress) external view returns (address) {
    address pool = s_pools[tokenAddress];
    if (pool == address(0)) revert
    PoolFactory__PoolDoesNotExist(tokenAddress);
    return pool;
}
```

[I-07] No zero-address check on `wethToken` in `PoolFactory` constructor**Description:**

The `PoolFactory` constructor assigns `wethToken` to the immutable `i_wethToken` without validating that it is not the zero address. Since `i_wethToken` is immutable, a deployment error cannot be corrected without full redeployment.

Proof of Concept:

```
function testPoolFactoryConstructorAllowsZeroWeth() public {
    // Deploys successfully with address(0) – should revert
    PoolFactory brokenFactory = new PoolFactory(address(0));
    assertEq(brokenFactory.getWethToken(), address(0));
    // Any pool created by this factory will inherit address(0) as WETH
}
```

Recommended Mitigation:

```
constructor(address wethToken) {
    if (wethToken == address(0)) revert PoolFactory__InvalidWethToken();
    i_wethToken = wethToken;
}
```

[I-08] `createPool()` does not validate that `tokenAddress` is a contract, allowing pool deployment for EOAs

Description:

`createPool()` accepts any address without verifying it has deployed contract code. Passing an EOA would cause `IERC20(tokenAddress).name()` to either revert or return empty data. Even if deployment somehow succeeded, the resulting pool would be permanently non-functional and would permanently occupy that address slot in `s_pools`.

Proof of Concept:

```
function testCreatePoolWithEOAAddress() public {
    address eoa = makeAddr("randomEOA");

    // This call will revert or produce a broken pool
    // because eoa has no contract code
    vm.expectRevert(); // .name() call on an EOA reverts
    factory.createPool(eoa);

    // Even if it didn't revert, the slot would now be permanently
    // occupied:
    // factory.getPool(eoa) != address(0)
    // blocking any future legitimate deployment for that address
}
```

Recommended Mitigation:

Add an explicit contract existence check:

```
if (tokenAddress.code.length == 0) revert
PoolFactory__NotAContract(tokenAddress);
```

Appendix — Vulnerability Summary

Severity	TSwapPool.sol	PoolFactory.sol	Total
High	4	0	4
Medium	2	0	2

Severity	TSwapPool.sol	PoolFactory.sol	Total
Low	3	1	4
Informational	5	3	8
Total	14	4	18